MetroMaya: How the Code Works

Nov 14, 2026 · @Sofia Agatha Atienza

Purpose

MetroMaya lets a business owner in Metro Manila see their permit fees and book a city hall appointment online, so they make one prepared trip instead of queueing several times. It is a course project for Object-Oriented Programming, built to show the four OOP pillars in a working system.

- **Who it's for:** owners and representatives of businesses in the 16 cities of Metro Manila and the municipality of Pateros.
- **Services:** a new business permit, an annual renewal, or an inspection visit.
- **The problem it solves:** applicants usually learn the fees, inspections and documents only at the counter. MetroMaya shows them first, then gives a reference number for a fixed time slot.
- **Guide:** Maya, a Eurasian tree sparrow dressed as a mail carrier, is the site's mascot.
- **Slogan:** "Skip the line, get on business."
- **Live site:** metro-maya.vercel.app. Source code: fortyy-tw0/MetroMaya on GitHub.

All fees are **sample rates for a course project, not official LGU fees**. The office addresses, barangays and ZIP codes are real; the money amounts are invented.

How it works for a user

A visitor learns the costs on the home page, books in five steps, and gets a reference number they use to track, move or cancel the booking. No account is needed.

Home page (`index.html`), top to bottom:

Section	What the visitor sees
Hero	The slogan, Book a slot / Estimate my fees, Maya peeking over a fact card
What is MetroMaya	Three reasons to book online
How It Works	The four booking steps
Fees ("What you pay")	Every type of business with its base fee; pick a city and a business size and all fees rescale

Section	What the visitor sees
Fees Estimator	Pick a service, type of business, employees and city; see the total, fee lines, inspections, time and documents
Offices	The chosen city's hall address and the four desks a permit passes through (BPLO, treasurer, BFP, health)
Closing card	"Ready to book your slot?" with a Book a slot button

Booking (`book.html`), five steps:

1. **Service:** new business permit, annual renewal or inspection visit.
2. **Business:** business name, Address Line 1, City → Barangay → ZIP code, type of business, employees, and the owner's name, email and mobile. A live estimate appears once type, employees and city are filled in.
3. **Office:** the business permits office of the city entered in step 2, already selected, with its real address.
4. **Date and time:** a weekday up to 30 days ahead; hourly slots 8:00 to 16:00, no 12:00 slot. Full slots are disabled.
5. **Review:** the summary, fee breakdown, inspections and documents to bring; Confirm returns a reference like `BPL-20261116-ABCDE`.

Track a booking (`manage.html`): enter the reference to see the booking, cancel it, or move it to another free slot.

Architecture

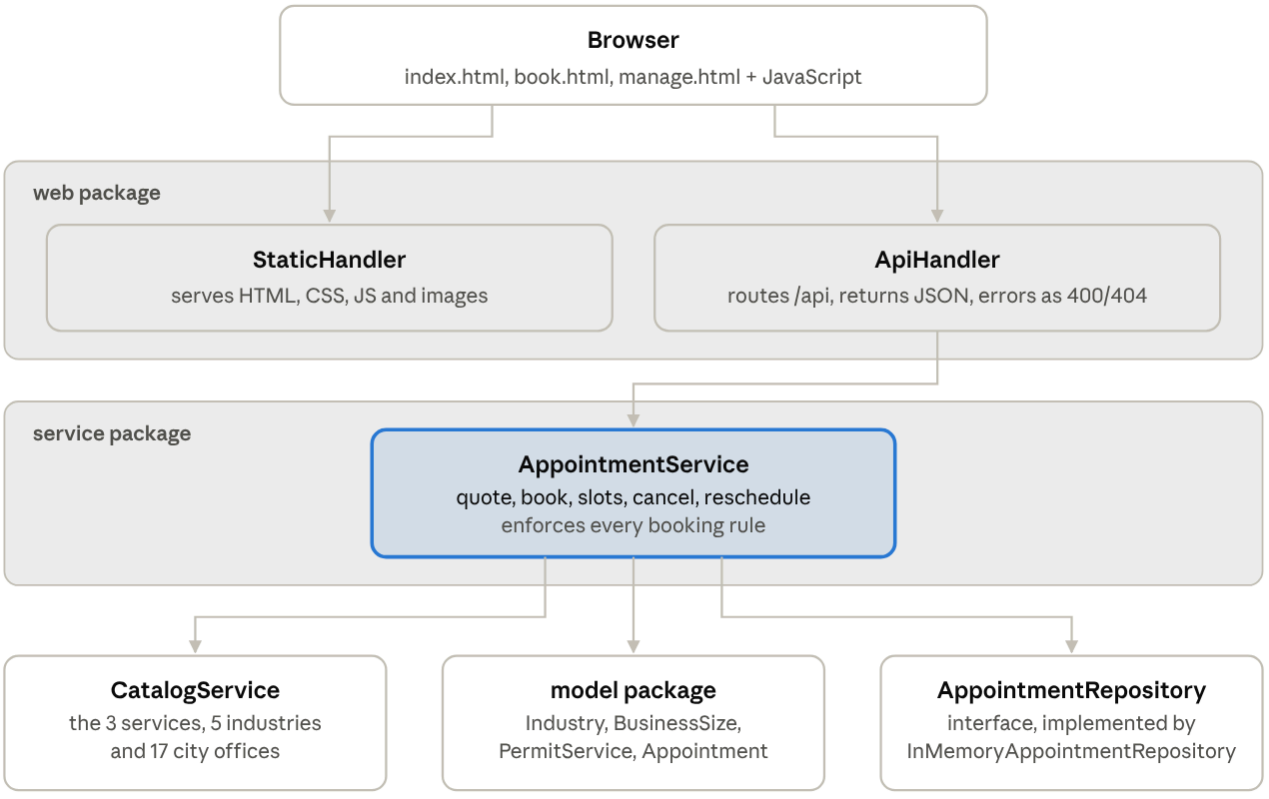
MetroMaya is one Java 21 program that both serves the web pages and answers a small JSON API; the browser does the rest in plain HTML, CSS and JavaScript. There are no frameworks, no build tool and no external libraries: the server uses the JDK's built-in `com.sun.net.httpserver`.

The Java code is split into four layers, each in its own package under `src/main/java/appointment/`:

Layer (package)	Job	Key classes
`web`	Receives HTTP requests, turns them into calls, writes JSON back	`WebServer`, `ApiHandler`, `StaticHandler`, `Json`

Layer (package)	Job	Key classes
`service`	Business rules: quoting, booking, slots, cancelling, rescheduling	`AppointmentService`, `CatalogService`, `SlotInfo`, `BookingException`, `NotFoundException`
`model`	The domain: industries, sizes, services, offices, fees, appointments	`Industry` + 5 subclasses, `BusinessSize`, `PermitService` + 3 subclasses, `Appointment`, `Business`, `Applicant`, `Office`, `FeeBreakdown`, `Quote`
`repository`	Stores appointments	`AppointmentRepository` (interface), `InMemoryAppointmentRepository`

A request passes from the browser through four layers



request flow · browser, web, service, model and repository

Page files go straight to `StaticHandler`; every `/api` call goes through `ApiHandler` to `AppointmentService`, which reads the catalog, works with the model objects and saves bookings through the repository interface.

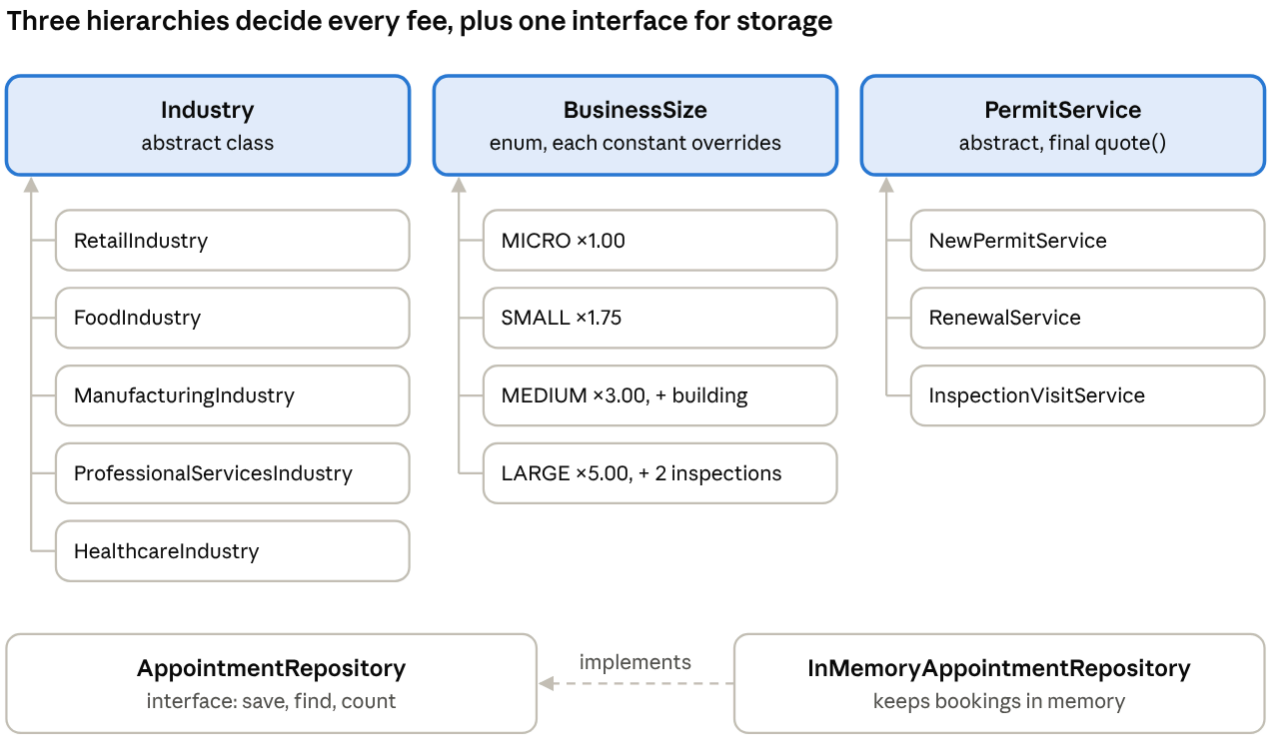
`Main` wires them together: it creates an `InMemoryAppointmentRepository`, a `CatalogService` and a clock set to Asia/Manila, hands them to `AppointmentService`, and starts `WebServer` on the port in the `PORT` environment variable (8080 by default).

Deployment: the root `Dockerfile` compiles the code with `javac` in a JDK 21 image, then runs it in a small JRE 21 image. Vercel builds that container on every push to `main` and serves it at metro-maya.vercel.app.

How the code uses OOP

All four pillars show up in the model, and three class hierarchies carry the polymorphism: what changes by **industry**, by **business size** and by **service** each lives in its own type, so the rest of the code never checks "which kind is this?".

Pillar	Where it shows	What it buys
Encapsulation	`private final` fields with getters in every model class; validation inside constructors (`Business` , `Applicant`); state changes only through `Appointment.cancel()` and `moveTo()`	An object can never be built or put in an invalid state
Abstraction	`abstract class Industry` , `abstract class PermitService` , `interface AppointmentRepository`	Callers depend on what an object can do, not how
Inheritance	5 subclasses of `Industry` , 3 subclasses of `PermitService` , shared helpers in the base class	Shared code is written once; each subclass adds only what differs
Polymorphism	Overridden methods in the subclasses and in each `BusinessSize` constant; the final template method `PermitService.quote()`	One call gives the right fee for any industry, size and service



class hierarchies · Industry, BusinessSize, PermitService and the repository

Arrows point from each subclass (or enum constant) to the type it specialises; the dashed arrow means the class implements the interface.

Encapsulation

Every model field is `private`, and most are `final`, so they are set once in the constructor and never change. Constructors check their input and throw a clear message, which the API returns to the user as-is:

```
public Business(String name, String address, Industry industry, int
employeeCount) {
    if (name == null || name.trim().length() < 2) {
        throw new IllegalArgumentException("Please enter your business
name.");
    }
    ...
}
```

`Appointment` is the one object whose state changes. Its `status`, `date` and `time` are private, and they change only through methods that enforce the rules: `cancel()` refuses a booking that's already cancelled, and `moveTo()` refuses to move a cancelled one. `FeeBreakdown` hides its list of lines and works out the total itself, so a total can't disagree with its lines.

Abstraction

`Industry` and `PermitService` are `abstract`: you can't create a plain "industry", only a concrete one such as `FoodIndustry`. They declare what every kind must answer (`requiredInspections()`, `computeFee(business)` and so on) without saying how.

`AppointmentRepository` is an interface with four methods: `save`, `findByReference`, `countActive` and `findActiveByEmail`. `AppointmentService` only knows the interface, so the in-memory store could be swapped for a database class without touching the booking rules.

Inheritance

Industries: `RetailIndustry`, `FoodIndustry`, `ManufacturingIndustry`, `ProfessionalServicesIndustry` and `HealthcareIndustry` all extend `Industry`. Each passes its id, name, description and base fee to the parent constructor through `super(...)`, and overrides what differs.

Industry	Base fee (sample)	Inspections	Extra charge
Retail & trading	₱1,500	Fire safety, Building & occupancy	none
Food & beverage	₱2,200	Sanitary & health, Fire safety	Sanitary permit fee ₱350
Manufacturing	₱3,500	Building, Fire safety, Environmental	Pollution control fee ₱800
Professional & personal services	₱1,200	Fire safety	none
Healthcare	₱3,000	Sanitary, Fire safety, Building	Health facility regulatory fee ₱700

Services: `NewPermitService`, `RenewalService` and `InspectionVisitService` extend `PermitService`. They inherit three protected helpers so the shared maths is written once: `allInspections()` (the industry's inspections plus the size's, no repeats), `addInspectionLines()` (each inspection's fee times the size's factor) and `scaledMinutes()`.

Polymorphism

1. Industry: overriding in subclasses. `surcharges()` has a default in `Industry` that returns no charges; only Food, Manufacturing and Healthcare override it. A fee calculation just calls `business.getIndustry().surcharges()` and gets the right answer for any industry.

2. Business size: an enum with constant-specific bodies. `BusinessSize` has four constants, and each one carries its own numbers and overrides the abstract methods `extraInspections()` and `extraDocuments()`:

Size	Employees	Fee multiplier	Inspection factor	Adds
MICRO	1–9	×1.00	×1.00	nothing
SMALL	10–99	×1.75	×1.25	nothing
MEDIUM	100–199	×3.00	×1.50	Building inspection; audited financial statements
LARGE	200+	×5.00	×2.00	Building + Environmental inspections; audited statements, certificate of occupancy

`BusinessSize.fromEmployees(n)` picks the tier, so no code anywhere switches on the size.

3. Service: the template method pattern. `PermitService.quote()` is `final`, so every service builds its quote in exactly the same order, while the steps it calls are abstract and differ per subclass:

```
public final Quote quote(Business business) {
    return new Quote(business.getSize(), computeFee(business),
inspectionsFor(business),
            requirementsFor(business), estimatedMinutes(business));
}
```

Step	New permit	Annual renewal	Inspection visit
`computeFee`	base fee × size multiplier, surcharges, inspections, ₱150 filing	base fee × 0.80 × size multiplier, surcharges, yearly inspections, ₱100 filing	inspections only, ₱100 dispatch

Step	New permit	Annual renewal	Inspection visit
`inspectionsFor`	all (industry + size)	only those marked recurring on renewal	all
`estimatedMinutes`	30 min scaled by size	20 min scaled by size	sum of the inspections' minutes, scaled

Composition and supporting types

Objects are built from other objects rather than inheriting from them: a `Business` **has an** `Industry`, and an `Appointment` **has an** `Applicant`, a `Business`, a `PermitService`, an `Office` and a `Quote`. The appointment keeps its quote as a snapshot, so later fee changes don't alter a booking already made.

- **Records** (`Quote`, `FeeLine`, `SlotInfo`) are small immutable value types.
- **Enums** (`InspectionType`, `AppointmentStatus`) give each inspection its label, fee, minutes and "repeats on renewal" flag, and each booking a CONFIRMED or CANCELLED status.
- **Custom exceptions** (`BookingException`, `NotFoundException`) let the web layer turn a broken rule into a 400 and a missing item into a 404.

A quote and a booking, step by step

A new-permit quote for a retail shop with 12 employees in Manila comes to **₱4,269.37**, and every number traces back to one class.

The quote (`GET /api/quote?serviceId=new-permit&industryId=retail&employees=12`):

1. `ApiHandler` reads the parameters and calls `AppointmentService.quote(...)`.
2. `CatalogService` finds the `NewPermitService` and the `RetailIndustry` objects by id.
3. `Business.forEstimate(retail, 12)` builds a nameless business; `BusinessSize.fromEmployees(12)` makes it **SMALL**.
4. `service.quote(business)`, the final template method, calls each step; `NewPermitService` supplies the fee, inspections, documents and minutes.
5. The browser multiplies each line by Manila's sample rate (1.15) and re-adds the total.

Fee line	Where it comes from	Base	In Manila (×1.15)
Business permit fee (small)	`RetailIndustry` ₱1,500 × `SMALL` 1.75	₱2,625.00	₱3,018.75
Fire safety inspection	`InspectionType.FIRE_SAFETY` ₱400 × 1.25	₱500.00	₱575.00
Building & occupancy inspection	`InspectionType.BUILDING` ₱350 × 1.25	₱437.50	₱503.12
Filing and processing fee	`NewPermitService` flat fee	₱150.00	₱172.50
Estimated total		**₱3,712.50**	**₱4,269.37**

The visit takes 30 minutes × 1.25 = **38 minutes** at the counter.

The booking (`POST /api/appointments`): `AppointmentService.book(...)` checks the rules in order and stops at the first one broken.

1. The office must offer the service and serve the business's size.
2. The date must be today or later, at most 30 days ahead, and a weekday.
3. The time must be one of the hourly slots (8:00–11:00, 13:00–16:00).
4. The slot must have room: `repository.countActive(office, date, time)` must be below the office's capacity.
5. The same email may not already hold an active booking for the same business and service.
6. A new `Appointment` is built with a fresh reference (`BPL-` + date + 5 random characters) and the quote as a snapshot, then saved.

A broken rule throws `BookingException` , which `ApiHandler` returns as HTTP 400 with the message, such as "Offices are closed on weekends. Please choose a weekday." An unknown reference throws `NotFoundException` , returned as 404. **Cancel** calls `appointment.cancel()` ; **reschedule** re-checks the new slot and calls `appointment.moveTo(date, time)` .

API reference

The front end talks to the server only through these JSON endpoints under `/api` ; `ApiHandler` routes them. Errors come back as `{"error": "<message>"}` with status 400 (a broken rule or bad input) or 404 (not found).

Method and path	Input	Returns
`GET /api/health`	none	`{status: "ok"}`
`GET /api/config`	none	`today`, `maxDaysAhead` (30), and `sizes[]`: id, label, range, feeMultiplier, inspectionFactor, extraInspections, extraDocuments
`GET /api/services`	none	`services[]`: id, name, description, baseDocuments
`GET /api/industries`	none	`industries[]`: id, name, description, baseFee, inspections, documents, surcharges
`GET /api/offices`	none	`offices[]`: id (the city id), name, address, serviceIds, sizesServed
`GET /api/quote`	`serviceId`, `industryId`, `employees`	`quote`: size {id, label, range}, fee {lines[{label, amount}], total}, inspections[{label, fee, minutes}], requirements[], estimatedMinutes
`GET /api/slots`	`officeId`, `date` (YYYY-MM-DD)	`slots[]`: time, capacity, remaining, available

Method and path	Input	Returns
`POST /api/appointments`	form fields: serviceId, officeId, date, time, businessName, businessAddress, industryId, employees, ownerName, email, phone	`appointment`: referenceNo, status, date, time, service, office, business and the quote
`GET /api/appointments/{ref}`	the reference	`appointment`
`POST /api/appointments/{ref}/cancel`	none	`appointment` with status CANCELLED
`POST /api/appointments/{ref}/reschedule`	`date`, `time`	`appointment` at the new slot

The API always quotes **base** sample fees; the browser applies the city rate.

Front end

The browser side is three static pages and five scripts in `src/main/resources/static/`, served as-is by `StaticHandler`; there is no framework or build step.

File	What it does
`index.html` + `js/landing.js`	Home page: fee estimator, the Fees section with city and size pickers, the Offices card
`book.html` + `js/book.js`	The five-step booking wizard and its live estimate
`manage.html` + `js/manage.js`	Look up, cancel or reschedule a booking by reference
`js/common.js`	Shared helpers and page-wide behaviour, used by every page
`js/ncr-places.js`	Barangays and ZIP codes for each city (generated, about 90 KB)
`css/style.css`	The one stylesheet; colours are tokens taken from Maya (navy cap, gold piping, brown outline)

Shared helpers in `common.js`:

- `el()` builds page elements safely, so text from the server can never inject markup.
- `api.get` / `api.post` call the server and turn an `error` reply into a message.
- `renderFee`, `renderInspections` and `renderDocs` draw the fee breakdown, inspections and "Documents to bring" the same way on every page.
- `NCR_CITIES` lists the 16 cities and Pateros, each with its hall, barangay and a **sample** rate (e.g. Manila 1.15, Makati 1.30, Pateros 0.85).
- `detectCity(text)` finds a city named in an address; `scaleQuote(quote, rate)` multiplies every fee line by the city rate and re-adds the total.

Page behaviour: sections fade in as you scroll down and out as you scroll back up; the header tabs jump to the centre of each section and highlight the one you're reading; the city and business-size choices are remembered in the browser.

Maya, the mascot: she flaps in the hero (a 16-frame strip played with CSS), follows the pointer with her eyes, winks in the estimator every 5 seconds, and takes off when you hover Book a slot. The frames are generated from the drawings by the Python scripts in `tools/maya/`, which bend the wings rather than animating cut-out parts.

Running it, data sources and limits

The project runs anywhere Java 21 is installed, with one command and nothing else to download.

Run locally: `./run.sh` compiles everything into `out/` and starts the site at `http://localhost:8080` (set `PORT` to change it). **Deploy:** push to `main`; Vercel builds the `Dockerfile` and publishes it.

Where the data comes from:

Data	Source	Real or sample
Fees, city rates, slot capacities	Written for this project	Sample
Barangays (1,706 in Metro Manila)	PSA's Philippine Standard Geographic Code list (npm `psgc` 2.2.0), with the 10 EMBO barangays moved from Makati to Taguig	Real
ZIP codes	PHLPost ZIP directory (npm `zipcodes-ph` 1.1.2)	Real
City hall and permit office addresses	City websites, DILG-NCR and news reports, gathered Nov 2026	Real, check before relying on them

`tools/places/build_places.py` regenerates `js/ncr-places.js` from the two data packages.

Limits:

- **Bookings live in memory.** They are lost whenever the server restarts or Vercel scales the container down. A database class that implements `AppointmentRepository` would fix this without changing the booking rules.
- **City rates are applied in the browser.** The server stores base fees, so the city-scaled amounts are shown consistently but not saved with the booking.
- **No automated tests** at the moment; changes are checked by building and running the site.
- **Vercel's image limit:** each deploy stores a ~66 MB image, and the project holds about 50. When it's full, old images must be deleted with `vercel vcr image rm` before builds succeed again.